Aim:- To compute MST using Prim's Algorithm.

Theory:- Prim's algorithm is a greedy algorithm used to find the minimum spanning tree (MST) of a connected, weighted, undirected graph. It starts from any vertex & repetedly adds the minimum-weight edge connecting a vertex in the MST to a vertex outside it, until all vertices are included.

Algorithm:-

```
PRIM (G, V, source)
1. FOR each vertex v;
       key[v] = INFINITY
       parent[v] = -1
       visited[v] = FALSE
2. key[source] = 0
3. REPEAT V times:
       u = unvisited vertex with minimum key[u]
       IF key[u] == INFINITY:
           PRINT "Graph is disconnected"
           STOP
       visited[u] = TRUE
       FOR each adjacent vertex v of u:
           IF visited[v] == FALSE AND
           weight(u,v) < key[v]:
           key[v] = weight(u,v).
           parent[v] = u
4. TotalCost = 0
```

Teacher's Signature

5. FOR each vertex v except source:
PRINT parent[v], v, key[v]
TotalCost = TotalCost + key[v]
6. PRINT TotalCost
7. STOP

C++ Program

```
#include <bits/stdc++.h>
using namespace std;
using pii = pair<int, int>;
const int INF = 1e9;
void prim(int V, vector<vector<pii>> &adj) {
  vector<int> key(V, INF), parent(V, -1);
  vector<bool> inMST(V, false);
  priority_queue<pii, vector<pii>, greater<pii>>
                pq;
key[0] = 0; pq.push({0, 0});
while (!pq.empty()) {
  int u = pq.top().second; pq.pop();
if (inMST[u]) continue;
inMST[u] = true;
for(auto [v, w] : adj[u])
  if(!inMST[v] && w < key[v])
  {
    key[v] = w;
parent[v] = u; pq.push({key[v],
int cost = 0;
for(int i = 1; i < V; i++)
{ cout << parent[i] << " - " << i << " weight " << key[i]
<< "\n"; cost += key[i]; }
```

Teacher's Signature

```
    cout << " Total cost : " << cost << "\n";
}
int main() {
    int V = 5; vector< vector<pii> > adj(V);
    adj[0] = {{1,2}, {3,6}};
    adj[1] = {{0,2}, {2,3}, {3,8}, {4,5}};
    adj[2] = {{1,3}, {3,7}};
    adj[3] = {{0,6}, {2,8}, {4,5}};
    adj[4] = {{1,5}, {2,7}, {3,9}}
    prim(V, adj);
    return 0;
}
```

Complexity

1. Time Complexity : O(E log V) using a priority queue.
2. Space Complexity : O(V+E) including the adjency list.

Viva Questions

Q1 What is Prim's Algorithm ?

Ans Prim's Algorithm is a greedy algorithm used to find the minimum spanning tree (MST) of a connected, weighted, undirected graph.

Q2 What is a minimum spanning Tree (MST)

Ans An MST connects all vertices with minimum total edge without forming any cycle.

Teacher's Signature _____

Q3 How does Prim's Algorithm selects edges?

Ans It repeatedly seeks the minimum-weight edge connecting a vertex already in the MST to a vertex outside it.

Q4 What is the role of a priority queue in a Prim's Algorithm?

Ans A priority queue helps select the vertex with the minimum key value efficiently.

Q5 What is the ~~minimum~~ time Complexity of Prim's Algorithm using a priority Queue?

Ans Its time Complexity is O(ElogV), where V is the number of vertices & E is the number of edges.

Date

Expt. No. 9. Page No. 38.

(Part-A)

Aim:- Print All nodes reachable from a given node in a diagraph using BFS method.

Theory:- Breadth-First search (BFS) is a graph traversal algorithm that visits vertices level by level, starting from a given source vertex. It uses a queue to store vertices & a visited array to avoid visiting the same vertex repeatedly. In a directed graph, it prints all vertices reachable from the starting vertex by following directed edges.

Algorithm:-

BFS(G, start, V)

1. Create a visited array & initilize all the vertices as FALSE.
2. Create an empty queue Q.
3. Mark start as visited
   Enqueue(Q, start).
4. WHILE Q is not empty:
   u = Dequeue(Q)
   PRINT u
   For each vertex v adjacent to u:
   IF visited[v] = FALSE:
   visited[v] = TRUE:
   Enqueue(Q, v)
5. STOP.

Teacher's Signature

C++ Program

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int V = 5;
    vector<vector<int>> adj[V];
    adj[0] = {1,2};
    adj[1] = {0,3};
    adj[2] = {0,4};
    adj[3] = {1};
    adj[4] = {2};
    vector<int> vis(V,0);
    queue<int> q;
    int start = 0;
    q.push(start);
    vis[start] = 1;
    while(!q.empty()) {
        int u = q.front();
        q.pop();
        cout << u << " ";
        for(int v : adj[u])
        if(!vis[v]) {
            vis[v] = 1;
            q.push(v); }
    }
    return 0;
}
```

Complexity

Time Complexity: O(V+E) using an adjency list

space Complexity : O(V).

Viva Question :-

Q1 What is BFS?

Ans BFS (Breadth first Search) is a graph traversal algorithm that visits vertices level by level, starting from a given vertex.

Q2 Which data structure is used in BFS?

Ans BFS uses a queue (FIFO) to process vertices in the order they are discovered.

Q3 What is the time complexity of BFS?

Ans Its time Complexity is O(V+E) when an adjacency list is used.

Q4 Why do we use a visited Array in BFS?

Ans It prevents vertices from being visited repeatedly & avoids infinite traversal in cyclic graphy.

Q5 Does BFS visited every vertex in a directed Graph?

Ans No, BFS visits only the vertices reachable from the given starting vertex by following directed edges.

Date

Expt. No. _____ Page No. 44

Part-B

Aim:- Check whether a given graph is connected or not using DFS.

Theory:- Depth-first search (DFS) is a graph traversal algorithm that explores a vertex & continues as deep as possible along each branch before backtracking. For an undirected graph, If all vertices are visited after performing DFS from any one vertex, the graph is connected, otherwise, it is disconnected.

Algorithm:-

```
DFS(G, u)
1. Mark vertex u as visited.
2. FOR each vertex v adjacent to u:
        IF visited[v] == FALSE;
            DFS(G, v)

CHECK_CONNECTED(G, V)
1. Initialize all vertices as unvisited.
2. Perform DFS(G, 0).
3. FOR each vertex v from 0 to V-1:
        IF visited[v] == FALSE:
            PRINT "Graph is disconnected"
            STOP
4. PRINT "Graph is connected".
5. STOP.
```

Teacher's Signature _____

Source Code:-

```
#include <bits/stdc++.h>
using namespace std;

// DFS traversal
void dfs(int u, vector<vector<int>> &adj,
         vector<int> &vis) {
    vis[u] = 1;
    for (int v : adj[u]) {
        if (!vis[v]) {
            dfs(v, adj, vis);
        }
    }
}
int main() {
    int V = 5;
    vector<vector<int>> adj(V);
    // Undirected Graph
    adj[0] = {1, 2};
    adj[1] = {0, 3};
    adj[2] = {0, 4};
    adj[3] = {1};
    adj[4] = {2};

    vector<int> vis(V, 0);
    dfs(0, adj, vis);
    bool connected = true;
```

```
for(int i =0; i<V; i++)
    {
        if(!vis[i]) {
            connected = false;
            break;
        }
    }
if(connected)
    cout << "Connected" <<endl;
else
    cout <<"Not Connected" <<endl;

return 0;
}
```

Complexity

Time Complexity = O(V+E)

Space Complexity = O(V)

Viva Question

Q1 What is DFS?

Ans DFS (Depth-first Search) is a graph traversal Algorithm that explores as deeply as possible before backtracking.

Q2 which data structure is used in DFS?

Ans DFS uses a stack or recursion to traverse the graph.

Teacher's Signature

Date

Expt. No. _____ Page No. 44

Q3. How do we check whether an undirected graph is connected using DFS?

A3. Start DFS from any vertex. If all vertices are visited, the graph is connected.

Q4. What is the time complexity of DFS?

A4. Its time complexity is O(V+E) using an adjacency list.

Q5. Why do we use a visited array in DFS?

A5. It prevents revisiting vertices to avoids infinite traversal in cyclic graphs.

Teacher's Signature _____

Date

Expt. No. 10. Page No. 45.

Aim:- To place N Queens on an N x N board so that no two queens attack each other.

Theory:- The N-Queens problem is a classic backtracking problem in which N Queens must be placed on an N x N chessboard so that no two queens attack each other. No two queens can come in the same row, column, or diagonal. Backtracking places queens column by column & removes a queen whenever a safe position cannot be found.

Algorithm

```
NQUEENS(board, col, N)
    if col == N
        print board
        return true

    for row = 0 to N-1
        if IS_SAFE(board, row, col, N)
            board[row][col] = 1

            if NQUEENS(board, col+1, N) == true
                return true
            board[row][col] = 0

    return false
```

Teacher's Signature

```
IS_SAFE (board, row, col, N)
    for i = 0 to col-1
        if board[row][i] == 1
            return false
    for i = row-1, j = col-1
    while i >= 0 and j >= 0
        if board[i][j] == 1
            return false
        i = i - 1
        j = j - 1

    for i = row+1, j = col-1
        while i < N and j >= 0
            if board[i][j] == 1
                return false
            i = i + 1
            j = j - 1

    return true.
```

Complexity

Time Complexity => $O(N!)$
(upper-bound estimate for the backtracking approach)
Space Complexity => $O(N^2)$
for the board, plus $O(N)$
recursion stack.

Source Code

```
#include <bits/stdc++.h>
using namespace std;
int N;
bool isSafe(vector<vector<int>>& board, int row,
            int col) {
    for (int j = 0; j < col; j++) {
        if (board[row][j] == 1)
            return false;
    }

    for (int i = row, j = col; j >= 0 && i >= 0; i--,
                                        j--) {

        if (board[i][j] == 1)
            return false;
    }
for (int i = row, j = col; i < N && j >= 0; i++, j--)
    { if (board[i][j] == 1)
        return false;
    }
    return true;
}

bool solveQueens(vector<vector<int>>& board,
                 int col) {

    if (col == N)
return true;
    for(int
```

```
for (int row = 0; row < N; row++) {
    if (isSafe(board, row, col)) {
        board[row][col] = 1;

        if (solveNQueens(board, col+1))
            return true;
        board[row][col] = 0;
    }
}
return false;
}

int main() {
    cout << "Enter value of N:";
    cin >> N;
vector<vector<int>> board(N, vector<int>(N,0));
    if (SolveNQueens(board, 0)) {
        cout << "\n Solution:\n";

for (int i = 0; i < N; i++)
    {
    for (int j = 0; j < N; j++)
        { cout << board[i][j] << " ";
    Cout << endl;
    }
else {
    cout << "No solution exists:\n"; }
return 0;
}
```

Teacher's Signature _____

Viva Question

Q1 What is the N-Queens Problem?

A1 The N-Queens Problem is placing N queens on an N x N chessboard so that no two queens attack each other.

Q2 What is back tracking.

A2 Backtracking is an algorithm technique that tries a solution & reverse the choice if it leads to an invalid result.

Q3 Why do we place queens column by column?

A3 It ensures that only one queen is placed in each column, reducing unnecessary checks.

Q4 What is the time complexity of the N-Queens problem?

A4 The commonly stated upper-bound estimate is $O(N!)$ for the backtracking approach.

Q5 What conditions make a queen's position safe?

A5 No other queen should be in the same row or on either diagonal. Since we place queens column by column, the current column is already empty.

Teacher's Signature _____